



INTELLIGENCE ARTIFICIELLE 2 : DL

TD-TP : Réseaux de Neurones & Deep Learning

DIPES 2 : 4ème année (S2)

Par **Stéphane C. K. TÉKOUABOU (PhD & Ing.)**

2025-2026

BP 47 Yaoundé, Cameroun.

TD-TP : Réseaux de Neurones & Deep Learning

Ce TP est constitué de 3 sous-parties assez simples à réaliser sur des jeux de données classiques mais qui vont nous permettre d'acquérir/renforcer ses compétences pratiques et consolider les notions déjà vues en cours ou dans les TPs précédents. Il nous conduira également vers la construction et le déploiement des modèles basés sur les réseaux de neurones et l'apprentissage profond (deep learning).

NB : Toutefois, nous essayerons de faire le lien avec les modèles d'apprentissage déjà abordés dans les TP précédents et que vous pourriez consulter ou vous référer au travail que nous avons réalisé l'année dernière et partagé via Deepnote [ICI](#).

Partie 1 : Utilisation des modèles simples

a- Analyse des données

Commençons par récupérer la base de données d'une campagne télémarketing comme indiqué. Nous allons la charger dans un dataframe pandas à l'aide de la fonction [read_csv](#).

Avec la bibliothèque [pandas](#), il existe plusieurs fonctions permettant une première analyse simple des données :

- L'attribut [shape](#) permet de connaître les dimensions du dataframe.
- La fonction [info](#) permet d'avoir un résumé rapide des données.
- La fonction [describe](#) permet d'avoir des statistiques sur différentes tendances sur les données.
- La fonction [head](#) permet d'afficher les premières lignes du dataframe.
- La fonction [value_counts](#) permet de compter le nombre d'occurrences de chaque valeur.
- ...

1) Utiliser ces fonctions pour répondre aux questions suivantes :

- Combien de classes ? Décrire le type d'apprentissage adapté à ce jeu de données.
- Combien de caractéristiques descriptives ? De quels types ?
- Combien d'instances ?
- Combien d'instances de chaque classe ? En déduire une discussion des métriques de performances permettant de construire et évaluer le modèle optimal pour ce problème.
- Comment sont organisés les instaces ?

- Construire la matrice de corrélations puis commenter les corrélations intra et inter variables.
- Pourrait-on avoir recours à la sélection des variables ? Justifier votre choix.

2) Croisez les variables deux à deux entre elles et tracer les nuages de points en utilisant la fonction `scatter_plot`. Tracer les droites de régression et donner toutefois les paramètres. Commentez les résultats obtenus.

b- Prétraitement des données

Préparation des données : effectuer le pre-processing nécessaire pour supprimer les valeurs manquantes, et remplacer les variables catégorielles par les “dummy variables” correspondantes. Séparer la base de données en une base d’apprentissage contenant 80% des données, et une base de test.

c- Implémentation des méthodes simples et évaluation

3) La bibliothèque `sklearn` fournit la fonction `GridSearchCV` parmi lesquelles des méthodes d’apprentissage simples (SVN, KNN, DT, NB, LR, ANN, ...) et les méthodes d’ensemble (RF, Bagging, GB, AdaBoost, ...). Entraîner ces différentes méthodes et comparer les performances (selon les métriques choisies en 1) obtenues par validation croisée (On pourra se servir de la recherche en grilles pour retrouver les paramètres optimaux des modèles tels que abordée dans le TP1 précédent)

d- Validation croisée :

Nous allons commencer par appliquer cette méthode de sélection des paramètres sur le cas des **arbres de décision** qui sont la base des méthodes d’ensemble.

1. Pour ce jeu de données, quel est le meilleur classifieur constant ?
Calculer son erreur de test, qui fournira un référentiel à battre.
2. Construire et visualiser (à l’aide de la fonction `graphviz` par exemple) un arbre de décision pas trop grand, calculer son erreur de test. En faisant varier les paramètres de construction de l’arbre (notamment *max-depth*) on obtient des résultats différents.
3. **Implémenter une validation croisée** pour sélectionner le ou les paramètres pertinent(s) pour l’algorithme *CART* sur ce jeu de données. On pourra se servir de la fonction `GridSearchCV` du package `sklearn.model selection`. Elle prend en entrée une méthode de classification et une grille de paramètres à tester, représentée par un dictionnaire. Par exemple si l’on veut tester les

profondeurs 5,10,15 et deux critères d'impuretés, on peut utiliser

```
param_grid = {  
    'max_depth': [5,10,15],  
    'criterion': ["entropy","gini"]  
}
```

4. Déterminer ainsi le meilleur paramètre de profondeur dans une grille bien choisie. On pourra visualiser l'erreur de validation croisée en fonction du paramètre de profondeur de l'arbre. Evaluer le classifieur optimisé ainsi obtenu.

e- Bagging et Random Forest :

La fonction *RandomForestClassifier* permet de construire un classifieur par la méthode des forêts aléatoires. Les paramètres cruciaux de cette méthode sont le nombre d'arbres agrégés (*n_estimators*, **B** dans le cours) et le nombre de variables prises en compte lors de chaque séparation (*max-features*, **p** dans le cours).

1. Comment se servir de la fonction *RandomForestClassifier* pour faire du Bagging ?
Implémenter le Bagging pour un nombre **B** d'arbres. Commenter la complexité et la performance de la méthode lorsque **B** augmente.
2. En choisissant une valeur pour **p**, construire maintenant un classifieur de type forêt aléatoire. Calculer son erreur *Out-Of-Bag* (qui vient en attribut du classifieur *clf* : *clf.oob_score*), et la comparer à l'erreur de test.
3. En se fixant une valeur de **B** raisonnable, effectuer une validation croisée sur le paramètre **p** pour construire un classifieur *Random Forest optimisé*.

e- Boosting :

On va expérimenter le Boosting à l'aide de la fonction *GradientBoostingClassifier*. Vérifier que vous disposez bien d'une version de *scikit-learn* supérieure à 0.20, car certaines fonctionnalités intéressantes ne sont disponibles qu'à partir de cette version.

- a- A partir de la documentation en ligne, identifiez les paramètres cruciaux de l'algorithme **Gradient Boosting**. Quels jeux de paramètres correspondent à l'algorithme **Adaboost** ?
- b- Le nombre total d'arbres agrégés **B** doit être sélectionné avec attention pour éviter le *sur-apprentissage*. Une implémentation de l' "*early stopping*" est possible dans *scikit-learn* à l'aide des paramètres *validation_fraction* et *n_iter_no_change*. À λ (taux d'apprentissage) et **p** (paramètre d'interaction) fixés, proposer un choix de **B**.

- c- Sélectionner par l'approche de votre choix un “*bon*” algorithme de *Gradient Boosting*, qui sera votre classifieur Gradient Boosting optimisé. Le comparer à l'*arbre de décision optimisé* et au classifieur *Random Forest optimisé* obtenus précédemment.

f- Sélection de variables :

Les méthodes d'agrégation d'arbres perdent le caractère interprétable d'un seul arbre de décision, dont on peut observer les coupes successives pour interpréter une décision. Toutefois, il est possible de déduire de toute méthode de construction ou d'agrégation d'arbres des “*variables importantes*” (dont la significativité statistique est moindre par rapport aux tests que l'on peut construire dans des modèles de régression).

- d- Les trois méthodes de classification implémentées disposent d'un attribut calculant un score d'importance pour les variables explicatives, [*clf.feature_importances*](#). Essayer de comprendre comment ces scores sont calculés, puis repérer les variables jugées importantes par chacun de vos classifieurs optimisés.

g- Evaluation : courbe ROC :

Les méthodes de classification qui produisent un score appartenant à plutôt qu'une classe peuvent être optimisées en choisissant un seuil potentiellement différent de 1 2 pour convertir le score en prédiction. A chaque seuil possible est associé un **taux de vrais positifs (TPR)** et un **taux de faux positifs (FPR)**. La qualité intrinsèque du score peut alors être évaluée via une courbe ROC, qui représente pour chaque seuil possible le TPR en fonction du FPR.

1. Comment prédire un score avec Random Forest ou Gradient Boosting ?
2. A l'aide de la fonction [*sklearn.metrics.roc_curve*](#), tracer sur le même graphique *une courbe ROC* pour les trois classifieurs optimisés obtenus précédemment.
3. Calculer l'**AUC (aire sous la courbe ROC)** pour ces trois classifieurs.

Partie 2 : Réseaux de neurones (Neural Networks)

Cette partie nous permettra d'implémenter notre premier réseau de neurones pour construire un modèle permettant prédire le succès de souscription d'un produit bancaire lors d'une campagne de télémarketing. Pour ce faire, nous allons travailler sur la base des **bank telemarketing** que vous pouvez télécharger [**ICI \(bank-additional\)**](#). Pour faciliter les étapes de traitement et se concentrer

sur la construction des réseaux de neurones, nous mettons à votre disposition le jeu de données prétraité avec le support de ce TP.

c- Modélisation d'un réseau de neurones artificiel

La bibliothèque Tensorflow fournit la fonction permettant de contruire votre propre réseau de neurone en suivant les étapes ci-après. On se servira particulièrement de la librairie [Keras](#) pour les réseaux de neurones avec une structure de chaîne (réseaux “feedforward”).

1. Créer votre model et attribuez lui un réseau de neurones vide (On utilisera model = Sequential(), couche dense et activation Softmax par exemple).
2. Visualiser l'architecture du réseau avec la méthode summary(),
3. Augmenter le nombre de couches puis compiler votre modèle.

Apprentissage et test

Déploiement du modèle

1. Enregistrez votre meilleur modèle sous format [.pkl](#) et sous le nom [telemarketing.pkl](#) pour la suite du déploiement.
2. Modifier le fichier de ploiement et déployer un modèle optimal construit avec les 10 variables indépendante les plus importantes.

Partie 2 : Apprentissage profond

Dans cette partie, nous utiliseron le jeu de données [Fashion MNIST](#) qui contient 70 000 images en niveaux de gris dans 10 catégories. Les images montrent des articles individuels de vêtements à basse résolution (28 par 28 pixels). Le jeu de données contient des images de chiffres écrits à la main (0, 1, 2, etc.) dans un format identique à celui des articles de vêtements que vous utiliserez ici. Le choix de ces deux ensembles de données (relativement petits) nous permettra de faire un premier pas vers l'entraînement d'un algorithme de reseaux de neurones profonds. Ceci nous servira de point de départ pour aborder des problèmes plus complexes tels que l'analyses d'images médicales ou toute autre problème similaire dans la partie 3.

Suivre les étapes suivantes pour entrainer votre modèle :

- from sklearn.neighbors import KNeighborsRegressor
- from sklearn.model_selection import train_test_split

— `from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, ...`

Afin d'évaluer les performances de nos modèles, nous allons introduire deux nouvelles mesures pour les problèmes de classification : **categorical_crossentropy**

Le modèle optimal résultera de l'homogénéité entre toutes ces trois mesures.

Entraînez d'autres architectures des réseaux de neurones profonds les plus populaires (LeNet, VGGNet, AlexNet, ZFNet, ResNet, Nasnet, DenseNet, ...) que vous importerez directement à partir des bibliothèques de python (tensorflow, keras) et comparer leurs performances avec le modèle précédemment construit.

Déploiement du modèle

1. Enregistrez votre meilleur modèle sous format `.pkl` et sous le nom `bank-tel.pkl` pour la suite du déploiement.
2. Modifier le fichier de ploiement et déployer un modèle optimal construit avec les 10 variables indépendante les plus importantes.

Réaliser votre TP1 et faites votre rapport que vous enverrez par mail avant le Dimanche 14 Avril 2026 à 23h59.

NB : Le rendu complet que vous enverrez par mail est un fichier `.zip` contenant :

- Le rapport du TP (en format pdf)
- le code python réalisé (fichier `.py` / `.ipynb` / Colab / ...)
- Le dossier de déploiement du modèle optimal

Ce travail comptera pour 30% dans la note finale de CC

Bibliographie